

Guia de Instalação — Automação SlimFit para uma nova franquia

Passo a passo para instalar o sistema completo (robô do Studio + Sofia + formulário) para **outra unidade/franquia**. Feito para quem vai fazer a instalação (você), não para o cliente final.

Tempo estimado: meio período a 1 dia, contando testes. A maior parte é configuração de contas e credenciais — o código já está pronto.

0. O que a franquia precisa ter (pré-requisitos)

Item	Para quê	Observação
VPS Ubuntu 22.04/24.04	Rodar o robô 24/7	2 vCPU / 4 GB RAM já servem. Ex.: Hostinger, Contabo, DigitalOcean
Conta EVO (W12) com login	Ler agenda, cadastrar, relatórios	E-mail + senha de um usuário com acesso
Conta/robô ZEE (opcional)	Fluxo automático ZEE → EVO	Tokens da API do ZEE
Número de WhatsApp dedicado	Envios do Studio	De preferência um chip só para isso
Conta Google + Service Account	Planilha de alunas	JSON da conta de serviço com acesso à planilha
Conta Render (grátis)	Hospedar o formulário	Deploy do app Flask
Chave da Anthropic (Sofia)	IA de atendimento	<code>ANTHROPIC_API_KEY</code>
Dados do Studio	Endereço, horários, preços, áudios das professoras, imagens	Para personalizar mensagens

 Cada franquia usa as **PRÓPRIAS credenciais**. Nunca reaproveite login de EVO/WhatsApp/Google de um Studio em outro.

1. Preparar o servidor (VPS)

Conecte por SSH (`ssh root@IP_D0_VPS`) e instale as dependências:

```
# fuso horário
timedatectl set-timezone America/Sao_Paulo
```

```
# Node.js 20 + ferramentas
apt-get update
apt-get install -y curl git python3 python3-venv python3-pip xvfb ffmpeg
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt-get install -y nodejs

# Chromium (para o Puppeteer/scraper)
apt-get install -y chromium-browser || apt-get install -y chromium

# PM2 (gerenciador de processos)
npm install -g pm2
```

Descubra o caminho do Chromium (vai no `.env` como `CHROMIUM_PATH`):

```
which chromium-browser || which chromium
```

2. Clonar o projeto e instalar

```
cd ~
git clone <URL_DO_REPOSITORIO> SF-Automacao
cd SF-Automacao/Experimental
npm install

# dependências da Sofia
cd ../ChatBot
npm install

# dependências do fluxo Python ZEE-EVO
cd ../Experimental/src/agendamento_evo
python3 -m venv .env
.env/bin/pip install -r requirements.txt # (ou os pacotes listados no projeto)
```

3. Configurar as credenciais (`.env`)

Crie o arquivo `Experimental/.env` (use os valores da franquia — os abaixo são **exemplos/placeholders**):

```
# EVO
EVO_URL=https://evo5.w12app.com.br
EVO_EMAIL=usuario@dominio.com
EVO_PASSWORD=SUA_SENHA_EVO
HEADLESS=false # scraper roda com tela (sob xvfb)
CHROMIUM_PATH=/usr/bin/chromium-browser

# Google Sheets
SHEETS_ID=ID_DA_PLANILHA
GOOGLE_SA_KEY=/caminho/para/service-account.json
SHEETS_ABA=Aniversarios
```

```
# Formulário / ponte com a nuvem
FORM_CLOUD_URL=https://SEU-FORM.onrender.com
FORM_OUTBOX_TOKEN=UM_TOKEN_SECRETO
STUDIO_OUTBOX_FILE=/root/SF-Automacao/Experimental/src/agendamento_evo/confirmacoes_outbox.json

# Grupo da equipe (nome exato do grupo no WhatsApp)
GRUPO_EQUIPE=SlimFit Equipe 🍷

# Instagram (deixe desligado até ter IP residencial/proxy)
IG_ENABLED=false
IG_USERNAME=usuario_instagram
IG_PASSWORD=senha_instagram
# IG_PROXY=host:porta # se for usar proxy residencial
```

E o `.env` do fluxo Python (`Experimental/src/agendamento_evo/.env`) com os tokens do ZEE/EVO.

Coloque o `service-account.json` no caminho indicado. **Confirme que `.env` e `service-account.json` estão no `.gitignore`** (já estão no projeto).

4. Conectar o WhatsApp do Studio (escanear o QR — 1 vez)

```
cd ~/SF-Automacao/Experimental
xvfb-run -a node src/wa-client.js
```

Vai aparecer um **QR Code** no terminal. No celular do número do Studio: **WhatsApp** → **Aparelhos conectados** → **Conectar um aparelho** → escaneie. Quando aparecer  **WhatsApp PRONTO**, a sessão está salva. Pode encerrar com `Ctrl+C`.

A sessão fica gravada em `webjs_auth/` — só precisa escanear de novo se o WhatsApp desconectar o aparelho.

5. Testar o EVO e os jobs (modo simulação)

Sempre que testar algo manualmente, **pare o robô** antes (para não haver dois usando a sessão) e religue depois:

```
pm2 stop slimfit-exp # se já estiver rodando

# testes em modo simulação (não enviam nada):
xvfb-run -a node src/resumo-dia.js --dry
xvfb-run -a node src/ausentes-10-dias.js --dry
xvfb-run -a node src/aniversariantes.js # simulação por padrão

pm2 start slimfit-exp
```

Ajuste o que for específico da franquia: **áudios das professoras** (pasta `audios/`), **nome do grupo**, **endereço/preços** nas mensagens.

6. Agendar o fluxo Python (ZEE → EVO)

Edite o `crontab` (`crontab -e`) e adicione (a cada 5 min):

```
*/5 * * * * /root/SF-Automacao/Experimental/src/agendamento_evo/rodar-vps.sh
```

(O script `rodar-vps.sh` ativa a venv e roda o job; ajuste os caminhos.)

7. Subir o robô com o PM2 (24/7)

```
cd ~/SF-Automacao/Experimental
pm2 start src/scheduler.js --name slimfit-exp --interpreter node \
  --node-args="" --
# se o scraper precisar de tela, use o wrapper com xvfb (scheduler-vps.sh)

pm2 save          # grava a lista de processos
pm2 startup        # faz o PM2 subir no boot (siga o comando que ele imprimir)
```

Confirme:

```
pm2 status
pm2 logs slimfit-exp --lines 40
```

No log devem aparecer as linhas `Job ... agendado` e `WhatsApp PRONTO`.

8. Formulário (Render) e Sofia

- **Formulário:** faça o deploy do app Flask no Render, configure as variáveis (chave do EVO, token do outbox) e aponte `FORM_CLOUD_URL` no `.env` do VPS para a URL do Render.
- **Sofia:** em `ChatBot/`, configure `ANTHROPIC_API_KEY` e o número do WhatsApp da Sofia; ajuste o **prompt** (tom, preço, endereço) pelo editor. Suba a Sofia com o PM2 também, quando o chip estiver pronto.

9. Checklist final de validação

- ☐ `pm2 status` mostra o processo **online**
- ☐ `WhatsApp PRONTO` no log
- ☐ Todos os `Job ... agendado` aparecem no log
- ☐ `resumo-dia.js --dry` lê EVO e monta os 5 blocos
- ☐ `ausentes-10-dias.js --dry` lista as alunas certas
- ☐ Formulário no ar e enviando para a fila
- ☐ Confirmação de teste chega no WhatsApp
- ☐ `pm2 save` executado (sobrevive a reboot)
- ☐ Áudios, nome do grupo, endereço e preços ajustados para a franquia

10. Operação e manutenção do dia a dia

Ação	Comando
Ver status	<code>pm2 status</code>
Ver logs ao vivo	<code>pm2 logs slimfit-exp</code>
Atualizar o código	<code>git pull</code> → <code>pm2 restart slimfit-exp</code>
Parar / iniciar	<code>pm2 stop slimfit-exp</code> / <code>pm2 start slimfit-exp</code>
Rodar um job na mão	<code>pm2 stop slimfit-exp</code> → <code>xvfb-run -a node src/<job>.js</code> → <code>pm2 start slimfit-exp</code>

Regra de ouro: script manual e PM2 **não** rodam ao mesmo tempo (compartilham a sessão do WhatsApp). Edições no servidor só valem após `pm2 restart`.

Dúvidas de instalação: seguir esta ordem (servidor → credenciais → WhatsApp → testes → PM2) evita 90% dos problemas. O que mais dá trabalho é a 1ª conexão do WhatsApp e as credenciais do EVO — teste esses dois primeiro.